GUIDE 54 · OPERATE

Install on Kubernetes with Helm

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=54-install-on-kubernetes-with-helm. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** QueryAPIGate installed by `helm install` - first one pod on SQLite with its store on a volume, then three pods on PostgreSQL and Redis with the events and MCP servers, changed while serving without a single failed request.

> **Stable** - the chart's values are under the compatibility promise since 0.30 (what that means). The server it installs is the same one as everywhere else.

Every command and output below was run on a **kind** cluster (Kubernetes 1.37) with **Helm 4.3**, using the chart and the image of the 0.26.0 release commit, built and loaded into the cluster before they were published. Helm 3 works the same; CI installs the chart on kind on every change.

The chart is published to GitHub's registry with each release, at the same version as the server:

```shell
helm show chart oci://ghcr.io/anantharajuc/charts/queryapigate --version 0.26.0
```

Step 1: One pod on SQLite

Keep the secrets in a Secret you manage (by hand here; sealed-secrets, External Secrets or Vault in production):

```shell
kubectl create namespace queryapigate
kubectl -n queryapigate create secret generic queryapigate \
  --from-literal=QUERYAPIGATE_API_KEY=<a long random key> \
  --from-literal=QUERYAPIGATE_SECRET_KEY=<a Fernet key>   # encrypts connection passwords in the store
helm install queryapigate oci://ghcr.io/anantharajuc/charts/queryapigate --version 0.26.0 \
  -n queryapigate --set existingSecret=queryapigate --wait
```

```text
STATUS: deployed
NOTES:
QueryAPIGate 0.26.0 is installed as queryapigate - one pod, its store on the volume queryapigate-data
(kept on uninstall).

See it's up:
  kubectl -n queryapigate port-forward svc/queryapigate 8080:80
  curl http://127.0.0.1:8080/health
then open http://127.0.0.1:8080/console/
```

```shell
kubectl -n queryapigate get pods,pvc
```

```text
pod/queryapigate-5fdf5f6b87-8g5tb   1/1     Running   0          4s
persistentvolumeclaim/queryapigate-data   Bound    pvc-1b36ccd9-...   1Gi        RWO            standard
```

After the port-forward, `/health` answers `{"status":"ok","version":"0.26.0",...}` and `/console/` opens the Console. Sign in with the API key from the Secret.

The pod runs locked down: as user 1000, with no capabilities, and with a read-only root filesystem. It can write only to `/data` (the volume) and `/tmp`:

```shell
kubectl -n queryapigate exec deploy/queryapigate -- sh -c 'id; touch /x'
```

```text
uid=1000(app) gid=1000(app) groups=1000(app)
touch: cannot touch '/x': Read-only file system
```

Step 2: What the chart refuses, and what it keeps

SQLite is one file on one volume, so asking for more pods is refused before anything changes:

```shell
helm upgrade queryapigate oci://ghcr.io/anantharajuc/charts/queryapigate --version 0.26.0 -n
queryapigate \
  --reuse-values --set replicas=3
```

```text
Error: UPGRADE FAILED: execution error at (queryapigate/templates/deployment.yaml:1:4): store.type
sqlite keeps the
store in one file on one volume - it runs one pod. For several, set store.type: postgres
(QUERYAPIGATE_DATABASE_URL
in the secret) and Redis.
```

A misspelt value is refused too. With `--set replica=3`, the values schema rejects it with `additional properties 'replica' not allowed`. The chart also refuses several pods without Redis, PostgreSQL without a database URL, and a secret setting (a key, the database or Redis URL) in `config`, where it would sit in plain text in the Deployment.

The volume outlives the release. With an API key `partner-a` created through the API, uninstall the release:

```shell
helm uninstall queryapigate -n queryapigate
```

```text
These resources were kept due to the resource policy:
[PersistentVolumeClaim] queryapigate-data

release "queryapigate" uninstalled
```

Then reinstall it with the same release name in the same namespace. It picks the volume up again, and `GET /api/v1/api-keys/partner-a` still answers `{"name": "partner-a", "active": true, ...}`. Delete the claim yourself when you really mean to drop the store.

Back up the SQLite store with `queryapigate backup`. It takes a consistent copy while the server keeps running:

```shell
kubectl -n queryapigate exec deploy/queryapigate -- queryapigate backup /tmp/backup.db --force
kubectl -n queryapigate cp <the pod>:/tmp/backup.db "./backups/queryapigate-$(date +%F).db"
kubectl -n queryapigate exec deploy/queryapigate -- rm /tmp/backup.db
```

```text
Backed up /data/queryapigate.db to /tmp/backup.db (225280 bytes)
```

To restore, scale the Deployment to 0. Then put the backup in place as `queryapigate.db` on the claim, from a one-off pod that mounts it, and remove the `-wal` and `-shm` files. Scale back to 1 afterwards. Back up and restore the store has the rest.

Step 3: Several pods on PostgreSQL and Redis

For several pods, the store goes to PostgreSQL and the rate limits, quotas and cache to Redis. Managed services are right for production. To try it, `deploy/kubernetes/dev-dependencies.yaml` starts throwaway ones:

```shell
kubectl create namespace prod
kubectl -n prod apply -f deploy/kubernetes/dev-dependencies.yaml
kubectl -n prod create secret generic queryapigate \
  --from-literal=QUERYAPIGATE_API_KEY=<a long random key> \
  --from-literal=QUERYAPIGATE_SECRET_KEY=<a Fernet key> \
  --from-literal=QUERYAPIGATE_DATABASE_URL=postgresql://queryapigate:
<password>@postgres:5432/queryapigate \
  --from-literal=QUERYAPIGATE_REDIS_URL=redis://redis:6379/0
```

`values-prod.yaml`:

```yaml
store: {type: postgres}
replicas: 3
existingSecret: queryapigate
redis: {inExistingSecret: true}   # the chart can't see inside your Secret: say Redis is there
events: {enabled: true}           # live events for many clients
mcp: {enabled: true}              # the MCP server for AI agents
config:                           # any other QUERYAPIGATE_* or OTEL_* setting
  QUERYAPIGATE_JSON_LOGS: "1"
```

```shell
helm install queryapigate oci://ghcr.io/anantharajuc/charts/queryapigate --version 0.26.0 \
  -n prod -f values-prod.yaml --wait
kubectl -n prod get deploy,pdb
```

```text
QueryAPIGate 0.26.0 is installed as queryapigate - 3 pod(s) on the PostgreSQL store.
...
The MCP server: http://queryapigate-mcp.prod.svc/mcp, in the cluster

deployment.apps/queryapigate          3/3     3            3           11s
deployment.apps/queryapigate-events   1/1     1            1           11s
deployment.apps/queryapigate-mcp      1/1     1            1           11s

poddisruptionbudget.policy/queryapigate   2               N/A              1                     11s
```

Each pod has registered itself in the shared store (`GET /api/v1/instances`):

```text
mcp    queryapigate-mcp-59747774d7-g2vd2     0.26.0
serve  queryapigate-666956f686-mfv92         0.26.0
events queryapigate-events-75c46d9d48-wjdg8  0.26.0
serve  queryapigate-666956f686-4dtkm         0.26.0
serve  queryapigate-666956f686-jq94t         0.26.0
```

A connection to the cluster's PostgreSQL and a saved query `tables`, both made through the API, answer from any pod:

```text
GET /q/tables
[{"table_name":"admin_sessions"},{"table_name":"admin_tokens"},{"table_name":"administrators"}]
```

Step 4: Change it while it serves

Add a setting to `values-prod.yaml` (`QUERYAPIGATE_HISTORY_RETENTION_DAYS: "30"` under `config`) and upgrade. A pod in the cluster called `/q/tables` every 50 ms throughout:

```shell
helm upgrade queryapigate oci://ghcr.io/anantharajuc/charts/queryapigate --version 0.26.0 -n prod -f
values-prod.yaml --wait
```

```text
Release "queryapigate" has been upgraded. Happy Helming!
REVISION: 2
```

```text
answered 1690, failed 0
```

The pods were replaced one at a time. With PostgreSQL, the chart never takes a pod down before its replacement is ready, and each pod finishes the requests it already has before stopping. Upgrading to the next release is the same command with the new `--version`. Upgrade one minor version at a time, and read its changelog entry first (Upgrade safely). CI runs this upgrade from the previous release to the current code with requests running, and fails on a single failed request.

One pod on SQLite is replaced, not rolled. The old pod stops before the new one starts, because both can't hold the volume. Expect a few seconds of downtime per upgrade.

Good to know

- **Changing the Secret doesn't restart the pods**: the chart can't see inside a Secret you manage. After rotating something in it, run `kubectl -n prod rollout restart deploy/queryapigate` (and `-events` and `-mcp` if you run them). A Secret the chart makes (`secret.create`), and anything in `config`, roll the pods on `helm upgrade`.
- **Ingress:** `ingress.enabled: true` with `ingress.host` routes `/events` to the events server and everything else to the API pods. `mcp.ingress.host` gives the MCP server its own host. The defaults carry ingress-nginx's no-buffering annotations for `/events`; with another controller, replace them with its own. CI checks the rendered routes. This run used port-forwarding instead of an ingress controller.
- **Files settings name by path** need a volume, because the root filesystem is read-only. The SIEM's CA and client certificate (guide 53), from a Secret:

  ```yaml
  extraVolumes: [{name: siem-tls, secret: {secretName: siem-client-tls}}]
  extraVolumeMounts: [{name: siem-tls, mountPath: /etc/queryapigate/siem, readOnly: true}]
  config:
    QUERYAPIGATE_AUDIT_SYSLOG_URL: tls://siem.example.com:6514
    QUERYAPIGATE_AUDIT_SYSLOG_CA: /etc/queryapigate/siem/ca.crt
    QUERYAPIGATE_AUDIT_SYSLOG_CERT: /etc/queryapigate/siem/tls.crt
    QUERYAPIGATE_AUDIT_SYSLOG_KEY: /etc/queryapigate/siem/tls.key
  ```

  The same goes for an export directory (`QUERYAPIGATE_EXPORT_DIRS`). CI mounts a file this way on kind and reads it back from the pod.
- **Prometheus Operator:** `serviceMonitor.enabled: true` scrapes every pod's `/metrics`. `networkPolicy.enabled: true` lets traffic in only from `networkPolicy.ingressFrom` and out only to `networkPolicy.egress` and DNS. List your databases there, or queries can't reach them.
- **Autoscaling** (`autoscaling.enabled: true`) needs the PostgreSQL shape.
- **Without Helm**, `deploy/kubernetes/queryapigate.yaml` is the same deployment as plain manifests. A test keeps it in step with what the chart renders.

Next steps

- All the chart's values, and Kubernetes in the deployment guide.
- Run several instances - what is shared between pods, and what isn't.
- Wire up Prometheus and Grafana.